

Docker Engine API – Hands-On Tutorial (WSL)

1. What is the Docker Engine API?

Docker is not magic. The **Docker CLI** (`docker run` , `docker ps` , etc.) is **just a client**.

Under the hood:

```
docker CLI → Docker Engine API → containerd / runc
```

- The **Docker Engine API** is a **REST API**
- It runs on the Docker daemon (`dockerd`)
- By default in Linux/WSL, it listens on a **Unix socket**

```
/var/run/docker.sock
```

When you run:

```
docker ps
```

It actually does something equivalent to:

```
GET /containers/json
```

2/38

2. Verify Docker API Socket in WSL

Inside WSL:

```
ls -l /var/run/docker.sock
```

Expected output:

```
srw-rw---- 1 root docker 0 docker.sock
```

Check daemon info using API:

```
curl --unix-socket /var/run/docker.sock http://localhost/_ping
```

Expected:

```
OK
```

If this works, your Docker API is accessible.

3/38

3. API Versioning

Docker API is versioned.

Check version:

```
curl --unix-socket /var/run/docker.sock http://localhost/version
```

Sample output:

```
{
  "Version": "25.0.3",
  "ApiVersion": "1.44"
}
```

We will use:

```
/v1.44/
```

4/38

4. List Containers (docker ps)

CLI

```
docker ps
```

API

```
curl --unix-socket /var/run/docker.sock \
http://localhost/v1.44/containers/json
```

All containers (including stopped):

```
curl --unix-socket /var/run/docker.sock \
"http://localhost/v1.44/containers/json?all=true"
```

5/38

5. Create a Container (docker create)

Step 1: Pull Image (docker pull)

```
docker pull nginx
```

API equivalent:

```
curl --unix-socket /var/run/docker.sock \
-X POST \
"http://localhost/v1.44/images/create?fromImage=nginx&tag=latest"
```

6/38

Step 2: Create Container

```
docker create --name mynginx -p 8080:80 nginx
```

API:

```
curl --unix-socket /var/run/docker.sock \
-X POST \
-H "Content-Type: application/json" \
http://localhost/v1.44/containers/create?name=mynginx \
-d '{
  "Image": "nginx",
  "ExposedPorts": {
    "80/tcp": {}
  },
  "HostConfig": {
    "PortBindings": {
      "80/tcp": [
        { "HostPort": "8080" }
      ]
    }
  }
}'
```

Response contains container ID.

7/38

6. Start / Stop / Restart Container

Start

```
docker start mynginx
```

API:

```
curl --unix-socket /var/run/docker.sock \  
-X POST \  
http://localhost/v1.44/containers/mynginx/start
```

Stop

```
docker stop mynginx
```

API:

```
curl --unix-socket /var/run/docker.sock \  
-X POST \  
http://localhost/v1.44/containers/mynginx/stop
```

Restart

```
curl --unix-socket /var/run/docker.sock \  
-X POST \  
http://localhost/v1.44/containers/mynginx/restart
```

8/38

7. Inspect Container (docker inspect)

```
docker inspect mynginx
```

API:

```
curl --unix-socket /var/run/docker.sock \
  http://localhost/v1.44/containers/mynginx/json
```

This gives:

- IP address
- Mounts
- Environment variables
- Resource limits

9/38

8. Container Logs (docker logs)

```
docker logs mynginx
```

API:

```
curl --unix-socket /var/run/docker.sock \
  "http://localhost/v1.44/containers/mynginx/logs?stdout=true&stderr=true"
```

Follow logs:

```
curl --unix-socket /var/run/docker.sock \
  "http://localhost/v1.44/containers/mynginx/logs?follow=true&stdout=true"
```

10/38

9. Execute Command Inside Container (docker exec)

CLI

```
docker exec -it mynginx bash
```

API (Two Steps)

Step 1: Create exec instance

```
curl --unix-socket /var/run/docker.sock \
  -X POST \
  -H "Content-Type: application/json" \
  http://localhost/v1.44/containers/mynginx/exec \
```

```
-d '{  
  "Cmd": ["ls", "/usr/share/nginx/html"],  
  "AttachStdout": true,  
  "AttachStderr": true  
}'
```

Response:

```
{ "Id": "exec_id_here" }
```

Step 2: Start exec

```
curl --unix-socket /var/run/docker.sock \  
-X POST \  
-H "Content-Type: application/json" \  
http://localhost/v1.44/exec/exec_id_here/start \  
-d '{ "Detach": false, "Tty": false }'
```

11/38

10. Remove Container (docker rm)

```
docker rm mynginx
```

API:

```
curl --unix-socket /var/run/docker.sock \  
-X DELETE \  
http://localhost/v1.44/containers/mynginx
```

Force remove:

```
curl --unix-socket /var/run/docker.sock \  
-X DELETE \  
"http://localhost/v1.44/containers/mynginx?force=true"
```

12/38

11. Images API

List Images

```
docker images
```

API:

```
curl --unix-socket /var/run/docker.sock \  
http://localhost/v1.44/images/json
```

Remove Image

```
docker rmi nginx
```

API:

```
curl --unix-socket /var/run/docker.sock \  
-X DELETE \  
http://localhost/v1.44/images/nginx
```

13/38

12. System & Stats

Docker Info

```
curl --unix-socket /var/run/docker.sock \  
http://localhost/v1.44/info
```

Container Stats (docker stats)

```
curl --unix-socket /var/run/docker.sock \  
http://localhost/v1.44/containers/mynginx/stats?stream=false
```

14/38

13. Why This Matters (Real Use Cases)

- Build **custom Docker dashboards**
- Write **CI/CD tools** (GitHub Actions, Jenkins)
- Build **desktop apps** (Go, Python, Electron)
- Control Docker **without shell access**
- Remote Docker management (with TLS)

14. Security Warning (Important)

Docker socket is **root equivalent**.

Anyone with access can:

- Start privileged containers
- Mount host filesystem
- Gain full host control

Never expose `/var/run/docker.sock` publicly.

15. Mini Practice Tasks (with hints)

Task 1

List only running containers using API.

Hint:

- Use `/containers/json`
- Look at `?all=false`

Task 2

Create an Ubuntu container that sleeps for 300 seconds.

Hint:

- Image: `ubuntu`
- Cmd: `["sleep", "300"]`

Task 3

Fetch logs of a stopped container.

Hint:

- Logs are available even after stop
- Use `stdout=true`

Task 4

Write a small script (curl or Go) that:

- Lists containers
- Prints name + state

Hint:

- Parse JSON from `/containers/json`